# Universiteti i Prishtinës “Hasan Prishtina”

## Fakulteti i Inxhinierisë Elektrike dhe Kompjuterike




## Dokumentim teknik i projektit

**Lënda: Menaxhimi i Burimeve në Cloud Computing**

## Titulli i projektit: Monitorimi i burimeve dhe observabiliteti i aplikacioneve në kontejnerë

| Profesori / asistenti | Emri dhe mbiemri i studentëve |
|---|---|
| Prof. Ass. Dr. Artan Mazrekaj | Adonis Xhemajli<br>Florian Saqipi<br>Gent Zhushi |

Prishtinë, 2026

# Përmbajtja

Abstrakti ..... 3

I. Përshkrimi i problemit ..... 4

1.1. Problemi dhe motivimi ..... 4

1.2. Qëllimi dhe objektivat ..... 4

1.3. Kufijtë e implementimit ..... 5

II. Teknologjitë e përdorura ..... 6

2.1. Teknologjitë dhe përgjegjësitë ..... 6

2.2. Konfigurimi i mjedisit punues ..... 7

2.3. Infrastruktura dhe arkitektura ..... 8

**2.4. Docker Compose dhe vendosja** ..... 9

III. Implementimi / Zhvillimi ..... 11

3.1. Organizimi i kodit dhe koordinimi ..... 11

3.2. Ngarkesat Compute dhe Data ..... 12

3.3. MongoDB, historiku dhe ndërfaqja ..... 13

3.4. Monitorimi dhe Grafana ..... 15

3.5. Gjurmimi i shpërndarë (Tracing) ..... 17

3.6. Alokimi i burimeve dhe replikat ..... 19

3.7. Eksperimentet dhe analiza e rezultateve ..... 21

3.8. Testimi, sfidat dhe kufizimet ..... 26

IV. Përfundime ..... 27

4.1. Arritjet kryesore ..... 27

4.2. Gjetjet dhe kufizimet ..... 27

4.3. Zhvillimi i ardhshëm ..... 27

V. References ..... 28

# Abstrakti

Ky projekt paraqet një laborator lokal për monitorimin e burimeve dhe observabilitetin e aplikacioneve në kontejnerë. Ideja kryesore është t'i lidhim ndryshimet e CPU-së, memories apo rrjetit direkt me veprimet që bën aplikacioni dhe t'i ruajmë rezultatet për analizë.

Sistemi ndërtohet me Docker Compose dhe shërbimet Flask Controller, Compute dhe Data. Controller koordinon një eksperiment aktiv, ndërsa ngarkesat ekzekutohen në procese të veçanta me kufinjë, afate dhe mundësi anulimi. MongoDB ruan të dhënat dhe mbështet ngarkesat e databazës. cAdvisor mbledh matjet e kontejnerëve; Prometheus ruan metrikat e burimeve dhe të aplikacionit; Grafana ofron dy dashboard-e për interpretimin e tyre. Gjurmimi opsional përmes OpenTelemetry Collector dhe Tempo lidh operacionet ndërmjet shërbimeve. Përmes konfigurimeve të testuara dhe ndarjes së llogaritjeve, mund të bëjmë krahasime të drejtpërdrejta me sasi fikse pune, duke përdorur një ose dy instanca Compute.

Katër eksperimentet shembull për CPU, memorie, rrjet dhe punë të kombinuar përfunduan dhe u ruajtën pa gabime operacionale të raportuara. Matjet treguan aktivitetin CPU, rritjen dhe lirimin e memories, drejtimin e trafikut dhe lidhjen me operacionet e databazës. Gjurma Combined që kemi ruajtur përmban 142 segmente të lidhura. Gjithashtu, historiku dhe eksporti në CSV ruajnë të dhënat reale, gjë që na ndihmon t'i verifikojmë më saktë grafikët.

Rezultatet demonstrojnë funksionimin e laboratorit në mjedisin lokal. Meqenëse këto janë testime me ngarkesa sintetike, nuk bëhet fjalë për një studim të plotë statistikor të performancës, por për një demonstrim praktik të sistemit. Përsëritja e eksperimenteve dhe vlerësimi i ndryshueshmërisë mbeten zhvillime për punën në të ardhmen.

**Fjalë kyçe:** monitorimi i burimeve, kontejnerë, Docker Compose, Prometheus, Grafana, gjurmim i shpërndarë.

# I. Përshkrimi i problemit

## 1.1. Problemi dhe motivimi

Aplikacionet e ndara në disa kontejnerë përdorin burime të përbashkëta të pajisjes ku ekzekutohen. Një shërbim mund të kryejë llogaritje, një tjetër të mbajë të dhëna në memorie, ndërsa një i tretë të lexojë nga databaza ose të dërgojë të dhëna përmes rrjetit. Edhe kur aplikacioni përgjigjet saktë, vetëm përgjigjja e tij nuk tregon sa CPU, memorie apo trafik rrjeti ka kërkuar për ta përfunduar punën. Për këtë arsye, funksionimi i aplikacionit duhet të shoqërohet me matje të burimeve që ai përdor.

Problemi i trajtuar në këtë projekt është ndërtimi i një mjedisi ku aktiviteti i kontrolluar i aplikacionit mund të lidhet me matjet reale të kontejnerëve. Kërkesa fillestare e laboratorit është një panel për monitorimin e CPU-së, RAM-it dhe rrjetit, duke përdorur Prometheus, Grafana dhe Flask. Për ta bërë demonstrimin e kësaj kërkese, projekti përfshin edhe shërbime që gjenerojnë ngarkesa të kufizuara. Kështu, ndryshimi në grafik mund të lidhet me një veprim të njohur, si kryerja e llogaritjeve ose mbajtja e një buffer-i në memorie.

Në një sistem me disa shërbime, interpretimi i matjeve kërkon të dihet edhe ku kryhet puna. Për shembull, gjatë një eksperimenti rrjeti, Data kërkon të dhëna nga Compute. Trafiku i pranuar pritet të rritet te Data, ndërsa trafiku i dërguar pritet të rritet te Compute. Një grafik i vetëm, pa dallimin e shërbimit dhe drejtimit të trafikut, do ta bënte më të vështirë shpjegimin e këtij aktiviteti. E njëjta nevojë paraqitet kur memoria e kontejnerit krahasohet me madhësinë e buffer-it të kërkuar ose kur përdorimi i CPU-së krahasohet me kuotën e lejuar.

Përveç matjeve të burimeve, është e dobishme të shihet çfarë po bën vetë aplikacioni. Numri i operacioneve, koha e ekzekutimit dhe gabimet ndihmojnë në interpretimin e ngarkesës. Gjurmimi i shpërndarë e plotëson këtë pamje duke lidhur operacionet e një eksperimenti ndërmjet shërbimeve. Në këtë raport, observabiliteti nënkupton pikërisht këtë mundësi për të kuptuar sjelljen e sistemit përmes matjeve dhe gjurmëve që ai prodhon.

## 1.2. Qëllimi dhe objektivat

Qëllimi kryesor është të ndërtojmë një laborator lokal ku mund të nisim eksperimente, të vëzhgojmë përdorimin e burimeve në kohë reale dhe të analizojmë rezultatet. Për këtë, përdorim Flask për ndërfaqen, Prometheus e Grafana për monitorimin, dhe cAdvisor për matjen e kontejnerëve, duke i mbajtur përgjegjësitë të ndara qartë.

Objektivat konkrete të projektit janë:

- **Matja e burimeve:** Monitorimi i saktë i CPU-së, memories dhe rrjetit në Grafana.
- **Gjenerimi i ngarkesave:** Krijimi i skenarëve të ndryshëm (CPU, memorie, rrjet, databazë, apo punë e kombinuar.
- **Kontrolli i ndërfaqes:** Një sistem i qëndrueshëm ku eksperimentet mund të nisen, ndalen apo pastrohen lehtësisht.

- **Ruajtja e të dhënave:** Regjistrimi i parametrave dhe eksportimi i tyre në CSV për analiza të mëvonshme.
- **Lidhja e metrikave:** Bashkimi i matjeve të burimeve me performancën e aplikacionit dhe gjurmët e shpërndara.
- **Krahasime të kontrolluara:** Vlerësimi i sjelljes së sistemit duke ndryshuar kufijtë e CPU-së ose duke përdorur dy instanca Compute.

Një rezultat i vlefshëm për laboratorin nuk është vetëm shfaqja e një grafiku. Duhet të mund të shpjegohet cili veprim e shkaktoi ndryshimin, në cilin shërbim u krye puna, cilët parametra u përdorën dhe çfarë rezultati u raportua. Ruajtja e këtyre të dhënave e bën demonstrimin të kontrollueshëm edhe pasi eksperimenti ka përfunduar.

## 1.3. Kufijtë e implementimit

Sistemi ekzekutohet në një laptop me Docker Desktop dhe kontejnerë Linux, duke imituar kështu konceptet e menaxhimit të burimeve në cloud, edhe pse gjithçka është e vendosur në nivel lokal. Matjet i bëjmë vetëm për kontejnerët e projektit, kështu që ato nuk pasqyrojnë konsumimin total të gjithë kompjuterit. Si konfigurim bazë kemi shtatë shërbime, të cilat bëhen nëntë nëse aktivizojmë sistemin e gjurmimit me OpenTelemetry Collector dhe Tempo, pa llogaritur këtu edhe një instancë shtesë opsionale që e përdorim për krahasime.

E gjithë puna është zhvilluar në pesë faza: fillimisht nisëm me monitorimin e thjeshtë të burimeve, vazhduam me kontrollin ndërveprues dhe historikun, pastaj shtuam metrikat e aplikacionit bashkë me gjurmimin, dhe në fund rregulluam alokimet, replikat dhe dashboard-et.

Për sa i përket ngarkesave, ato janë sintetike dhe të kufizuara qëllimisht për të krijuar një aktivitet të dallueshëm, e jo për të simuluar një aplikacion prodhimi apo trafik real përdoruesish. Vlerat që vendosim (si intensiteti i CPU-së apo numri i kërkesave) janë thjesht hyrje të eksperimentit, ndërkohë që rezultatet që shohim mund të ndikohen pak edhe nga vetë sistemi operativ apo shërbimet e monitorimit.

Katër eksperimentet që kemi ruajtur (për CPU, memorie, rrjet dhe punë të kombinuar) janë thjesht ekzekutime individuale për secilin skenar. Edhe krahasimet që bëjmë shërbejnë për të parë sjelljen e sistemit në ato kushte konkrete, por nuk mjaftojnë për të nxjerrë përfundime të plota statistikore për performancën – një studim i tillë i detajuar ngelet për t'u bërë në të ardhmen. Në kapitujt e tjerë vijues do të shpjegojmë më gjerësisht teknologjitë, mënyrën e implementimit dhe rezultatet e bazuara te të dhënat që kemi ruajtur [1, 15].

# II. Teknologjitë e përdorura

## 2.1. Teknologjitë dhe përgjegjësitë

Në laborator përdorim një grup teknologjish me përgjegjësi të ndara qartë: Docker Compose menaxhon shërbimet dhe lidhjet e tyre, Flask drejton aktivitetin, kurse mjetet e observabilitetit e bëjnë atë të matshëm, gjë që na lejon të krahasojmë drejtpërdrejt rezultatet e aplikacionit me konsumin real të kontejnerit [1, 2].

Në tabelën 1 janë përmbledhur teknologjitë kryesore dhe versionet e konfigurimit ose të mjedisit. Për të garantuar riprodhueshmërinë e mjedisit dhe për të dalluar ndryshimet e kodit nga ndryshimet e burimeve të lejuara, imazhet e infrastrukturës ruajnë identifikuesit *digest* në Docker Compose, ndërsa libraritë Python kanë versione të fiksuara. [1]

Tabela 1: Teknologjitë kryesore të laboratorit

| Teknologjia | Versioni në projekt | Përgjegjësia |
|---|---|---|
| Docker Compose | 5.1.4, plugin lokal | Nis dhe menaxhon shërbimet, rrjetin, volumet dhe profilet. |
| Python / Flask / Gunicorn | 3.13 / 3.1.3 / 26.2.0 | Ekzekutojnë Controller, Compute dhe Data; ndajnë kërkesat HTTP nga puna në sfond. |
| Jinja2 / HTML / CSS / JavaScript | Jinja2 3.1.6 | Ndërtojnë ndërfaqen dhe përditësimin periodik të progresit, pa framework tjetër frontend. |
| MongoDB / PyMongo | 8.0 / 4.19.0 | Ruajnë historikun dhe kryejnë operacionet e ngarkesave të databazës. |
| cAdvisor | 0.60.6 | Ekspozon konsumin real të CPU-së, memories dhe rrjetit të kontejnerëve. |
| Prometheus / klienti Python | 3.15.1 / 0.26.0 | Mbledhin e ruajnë metrikat e burimeve dhe të aplikacionit. |
| Grafana | 13.3.0 | Paraqet dashboard-et dhe hap gjurmët nga burimi Tempo. |
| OpenTelemetry SDK / Collector | 1.45.2 / 0.164.0 | Instrumentojnë operacionet dhe përcjellin gjurmët përmes OTLP/HTTP. |
| Grafana Tempo | 3.1.1 | Ruan gjurmët dhe i bën të kërkueshme nga Grafana. |

Për sa i përket funksionimit, Flask shërben ndërfaqen dhe API-të, ndërsa Gunicorn ekzekuton çdo aplikacion me një proces aplikativ dhe tetë fije për kërkesat, duke i mbajtur proceset e përkohshme të ngarkesave të ndara në mënyrë që të koordinohet puna dhe të lejohen kërkesat për gjendjen apo ndalimin e saj në çdo kohë [1, 3].

Më pas, sa i përket të dhënave, MongoDB, Prometheus dhe Tempo kujdesen për ruajtjen e llojeve të ndryshme të informacioneve (përmbledhjet e eksperimenteve, seritë kohore dhe gjurmët), Grafana i lexon ato përmes burimeve të veta, kurse versionet e listuara në projekt pasqyrojnë vetëm konfigurimin e këtij laboratori dhe nuk jepen si rekomandim për versione më të reja [4, 6, 14].

## 2.2. Konfigurimi i mjedisit punues

Mjedisi ku kemi bërë testimet është një laptop me Windows 11 Pro dhe Docker Desktop me motor Linux, ku kontejnerët ekzekutohen brenda mjedisit Linux ndërsa CPU-ja dhe memoria mbeten burime të përbashkëta të vetë kompjuterit. Për nisjen dhe verifikimin e gjithçkaje përdorim skripta PowerShell drejtpërdrejt nga baza (root) e repository-t.

Tabela 2: Mjedisi punues i përdorur

| Elementi | Konfigurimi i inspektuar |
|---|---|
| Sistemi operativ | Windows 11 Pro, versioni 10.0.26200 |
| Procesori | AMD Ryzen 7 4800H; 8 bërthama, 16 procesorë logjikë |
| Memoria fizike | Rreth 16 GB RAM |
| Docker Engine / Compose | 29.5.2 / 5.1.4 |
| PowerShell | 7.6.5 |
| Mjedisi i aplikacionit | Imazh Python 3.13, slim-bookworm; varësitë instalohen brenda imazheve |

Kufijtë fillestarë të kontejnerëve paraqiten në Tabelën 3. Ata tregojnë tavanin e tyre në bërthama CPU dhe memorie në MiB, por këto vlera nuk janë konsum i garantuar dhe nuk e rezervojnë automatikisht të gjithë sasinë e shënuar.

Tabela 3: Kufijtë e burimeve për konfigurimin bazë

| Shërbimi | CPU | Memoria |
|---|---|---|
| Controller | 0.5 | 256 MiB |
| Compute | 1 | 512 MiB |
| Data | 1 | 512 MiB |
| MongoDB | 1 | 768 MiB |
| cAdvisor | 0.5 | 256 MiB |
| Prometheus | 1 | 768 MiB |
| Grafana | 0.5 | 512 MiB |
| Collector — vetëm profili tracing | 0.5 | 256 MiB |
| Tempo — vetëm profili tracing | 1 | 768 MiB |
| Compute-2 — vetëm preset-i replica-2 | 1 | 512 MiB |

Kështu, shuma e tavaneve të memories llogaritet në 3.5 GiB pa gjurmim dhe 4.5 GiB me gjurmim (pa llogaritur instancën Compute-2), ndërsa preset-et më të mëdha arrijnë deri në 5 GiB, duke kërkuar gjithashtu hapësirë shtesë për Windows-in dhe Docker-in. Ndërkohë, shkarkimi dhe ndërtimi i parë kërkojnë lidhje interneti, por më pas mund të përdoren imazhet lokale, kurse për arsye sigurie, kredencialet e Grafana-s ruhen në skedarin lokal .env i cili përjashtohet nga Git.

## 2.3. Infrastruktura dhe arkitektura

Si pikë hyrëse për përdoruesin shërben Controller, i cili shfaq ndërfaqen, pranon parametrat, koordinon eksperimentet aktive dhe ruan përmbledhjet në MongoDB, ndërkohë që Compute merret me llogaritjet e CPU-së dhe dërgon përgjigjet binare për eksperimentet e rrjetit, Data mban buffer-in e memories, tërheq payload-e nga Compute dhe ekzekuton ciklet e lexim/shkrimit në MongoDB, dhe Compute-2 përdor të njëjtin imazh por dallohet veçmas kur aktivizohet krahasimi me dy instanca.

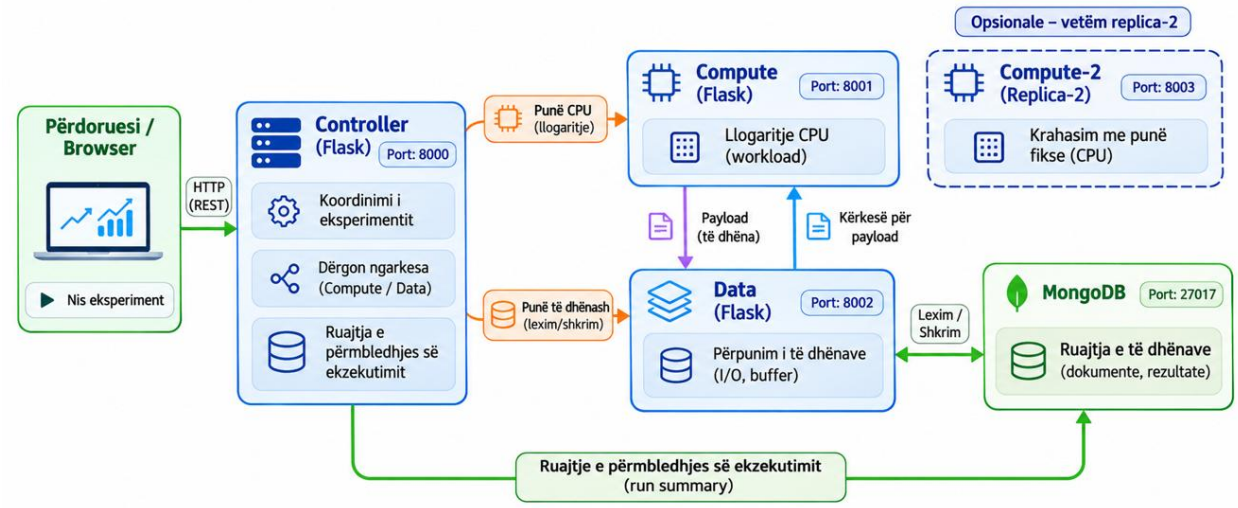



Figura 1: Arkitektura e shërbimeve të aplikacionit

cAdvisor vëzhgon kontejnerët për burimet kurse Prometheus i lexon çdo pesë sekonda, Prometheus lexon direkt endpoint-in */metrics* të shërbimeve për metrikat e aplikacionit, kurse për gjurmët aplikacionet i dërgojnë ato te Collector që i përcjell te Tempo, prej ku Grafana i hap për eksperimentin përkatës [5, 6, 13, 14].

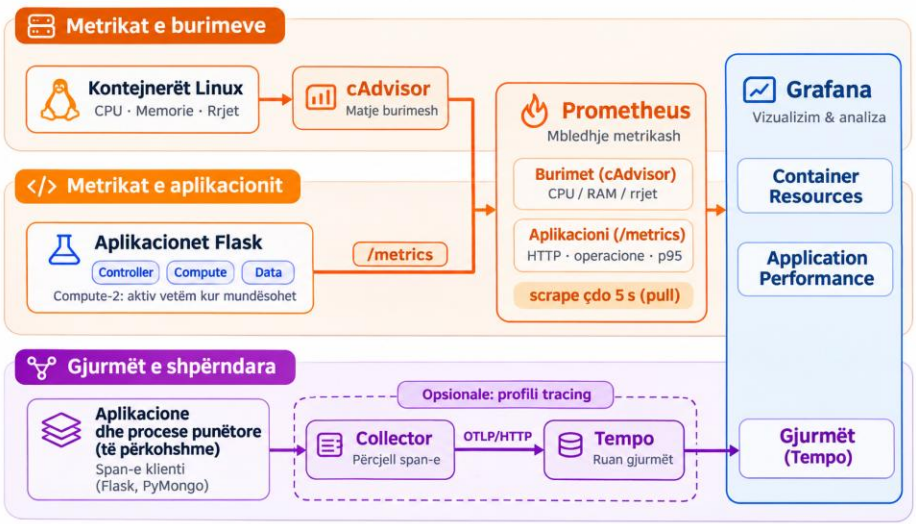



Figura 2: Rrjedhat e metrikave dhe gjurmëve

Kështu, pa sistem gjurmimi kemi shtatë kontejnerë, me profilin *tracing* bëhen nëntë, kurse preset-i *replica-2* shton edhe Compute-2 duke çuar numrin në tetë ose dhjetë kontejnerë. Për më tepër, cAdvisor ka qasje të privilegjuar në mjedisin Linux të Docker-it për matjet, ndërsa aplikacionet Flask nuk marrin *Docker socket* dhe nuk ndryshojnë vetë alokimin e burimeve, pasi ndryshimet e vendosjes bëhen gjithmonë përmes skriptave të projektit.

## 2.4. Docker Compose dhe vendosja

Docker Compose lidh të gjitha shërbimet së bashku në rrjetin *bridge* të projektit duke përdorur emra si compute ose mongodb, ndërsa në pajisjen lokale publikohen vetëm Controller tek porti 5000, Grafana tek 3000, Prometheus tek 9090 dhe MongoDB tek 27017 për t'u inspektuar përmes Compass, duke bërë që të katër portet të lidhen me 127.0.0.1 ndërsa endpoint-et e punës për Compute dhe Data të mos publikohen fare në host.

```
name: cloud-lab1

services:
  compute:
    build: <2 keys>
    image: cloud-lab1-compute:stage4
    environment: <2 keys>
    init: true
    cpus: 1.0
    mem_limit: 512m
    pids_limit: 64
    networks: [lab]
    restart: unless-stopped
    stop_grace_period: 10s
    healthcheck: <5 keys>

  data:
    build: <2 keys>
    image: cloud-lab1-data:stage4
    init: true
    cpus: 1.0
    mem_limit: 512m
    pids_limit: 64
    environment: <4 keys>
    networks: [lab]
    restart: unless-stopped
    stop_grace_period: 10s
    healthcheck: <5 keys>
```

Figura 3: Konfigurimi i shërbimeve në Docker Compose

Për nisjen përdoret skripta e projektit e cila zgjedh preset-in dhe profilin, kontrollon konfigurimin, pret gatishmërinë dhe verifikon kufijtë efektivë, duke bllokuar ndryshimet e vendosjes gjatë një eksperimenti aktiv. Ndërkohë, shërbimet me *healthcheck* kontrollohen nga vetë Docker, kurse për Collector dhe Tempo skriptat verifikojnë endpoint-et e gatishmërisë, megjithëse një *scrape* i suksesshëm nuk provon vetvetiu se eksperimenti ka dalë mirë.

Për konfigurimin bazë me gjurmim përdoren komandat:

```
./scripts/start.ps1 -Preset baseline -Tracing
./scripts/verify.ps1
```

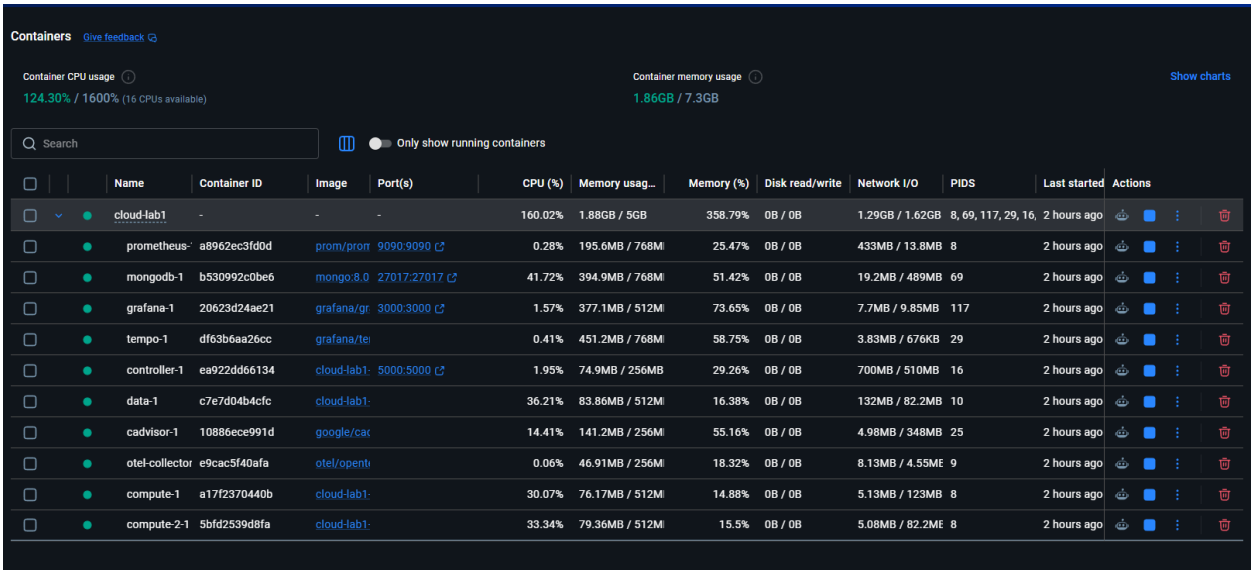



Figura 4: Dhjetë kontejnerët me gjurmim dhe instancën Compute-2

Për më tepër, volumet e emërtuara kujdesen që gjatë një mbylljeje normale të ruajnë historikun e MongoDB-së, metrikat e Prometheus-it, konfigurimin e Grafana-s dhe gjurmët e Tempo-s (ku Prometheus ruan të dhëna për shtatë ditë ose 1 GiB kurse Tempo ruan gjurmët për rreth 24 orë), me konfigurime monitorimi që montohen vetëm për lexim. Së fundmi, mbyllja normale kryhet me ./scripts/stop.ps1 pa i fshirë këto volume, pasi fshirja e të dhënave është një operacion plotësisht i veçantë [1, 7].

# III. Implementimi / Zhvillimi

## 3.1. Organizimi i kodit dhe koordinimi

Në anën e implementimit, kemi ndarë qartë pjesën e pranimit të kërkesës nga vetë ekzekutimi i punës, ku Controller merret me leximin e parametrave nga ndërfaqja, validimin e tyre dhe krijimin e rekordit të eksperimentit, ndërsa shërbimet Compute ose Data ekzekutojnë ngarkesën në procese të veçanta dhe raportojnë progresin në vazhdimësi. Kjo ndarje na ndihmon jashtëzakonisht shumë që kërkesat për gjendjen e sistemit apo butoni "Stop" të mund të përpunohen pa problem edhe gjatë kohës që serveri po kryen punë intensive [1].

Tabela 4: Blloqet kryesore të implementimit

| Moduli ose grupi | Përgjegjësia në rrjedhën e punës |
|---|---|
| controller: app.py | Faqja, API-të publike, filtrat e historikut dhe eksporti CSV. |
| coordinator.py / fixed_coordination.py | Pronësia e eksperimentit aktiv, shpërndarja, mbledhja e rezultateve dhe rikuperimi. |
| shared/model.py | Preset-et, validimi dhe paraqitja publike e rekordit me lidhjet për Grafana. |
| shared/worker.py / workload_child.py | Mbikëqyrja e procesit të përkohshëm dhe ngarkesat me kohëzgjatje. |
| fixed_work.py / fixed_runner.py / fixed_child.py | Caktimi i batch-eve, llogaritja deterministe dhe mbikëqyrja e disa proceseve. |
| repository.py / metrics.py / tracing.py | Ruajtja MongoDB, bashkimi i progresit në metrika dhe lidhja e gjurmëve. |

Kur përdoruesi klikon Start, vlerat e profilit bashkohen me ndryshimet e avancuara dhe ruhen si konfigurim i përfunduar, ku Controller kontrollon paraprakisht gatishmërinë, kufijtë e memories dhe gjendjen e vendosjes përmes një mekanizmi kyçjeje që mbron eksperimentin aktiv. Vetëm pas këtyre kontrolleve krijohet ID-ja dhe kthehet përgjigjja HTTP 202, pa qenë nevoja që ndërfaqja të presë mbylljen e ngarkesës.

Tabela 5: API-të kryesore të përdorura nga ndërfaqja

| Kërkesa | Sjellja kryesore |
|---|---|
| GET /api/workloads | Jep profilet, kufijtë dhe disponueshmërinë e ngarkesave. |
| POST /api/runs | Nis eksperimentin në mënyrë asinkrone; konfliktet aktive kthejnë 409. |
| GET /api/runs/active | Jep eksperimentin aktiv ose vlerë bosh. |
| POST /api/runs/<id>/stop | Kërkon anulimin; përsëritja nuk nis veprim tjetër. |
| GET /api/runs dhe /api/runs/export.csv | Lexojnë historikun dhe eksportojnë përmbledhjet sipas filtrave. |

Gjatë ekzekutimit, rekordi kalon nga starting në running dhe në stopping, duke përfunduar si completed, cancelled, failed ose interrupted. Përpjekjet me ID ekzistuese dhe parametra të ndryshuar refuzohen, ndërsa Controller ruan rezultatin pa e përsëritur automatikisht punën.

## 3.2. Ngarkesat Compute dhe Data

Compute trajton ngarkesat e CPU-së, kurse Data menaxhon memorien, rrjetin dhe databazën, duke përdorur profilin *Medium* dhe kohëzgjatjen prej 30 sekondash si pikënisje, ndërsa profilet *Low* ose *High* ndryshojnë vetëm intensitetin e jo kuotën e kontejnerit, e cila ndryshohet përmes preset-eve të vendosjes.

Tabela 6: Ngarkesat dhe konfigurimi fillestar Medium

| Ngarkesa | Puna e kryer dhe vlera fillestare | Kufizimi ose rezultati kryesor |
|---|---|---|
| CPU | Një proces alternon punë dhe pushim; 50% kohë aktive, 30 s. | Raporton ciklet llogaritëse dhe kohën reale. |
| CPU comparison | 32 batch-e me nga 2 500 njësi; dy procese gjithsej. | Përfundon me punën e caktuar; afat sigurie 180 s. |
| Memory | Alokon e prek 64 MiB dhe e mban buffer-in për 30 s. | Dalja e procesit liron buffer-in. |
| Network | Data shkarkon përgjigje 256 KiB nga Compute, me synim 5/s. | Numëron përgjigjet e plota dhe byte-t e lexuar. |
| Database | Shkruan dhe lexon dokument 16 KiB; synimi 5 cikle/s. | Një cikël ka një shkrim dhe një leximi të suksesshëm. |
| Combined Data | Mban 64 MiB dhe kryen punën Network dhe Database, me synim 5 cikle/s. | Raporton veçmas memorien, rrjetin, databazën dhe gabimet. |

Gjatë ekzekutimit, numëruesi cpu_batches regjistron llogaritjet elementare, ndërsa te memoria buffer-i preket në hapa prej 4 KiB që të përdoret realisht e jo vetëm si hapësirë virtuale.

Për rrjetin, përmbajtja lexohet në pjesë dhe kërkesa numërohet e suksesshme vetëm kur plotësohet i gjithë *payload*-i, ndërsa databaza ripërdor deri në 128 pozicione dokumentesh dhe verifikon përputhjen mes leximit dhe shkrimit.

Validimi pranon kohëzgjatje 15–180 sekonda, intensitet CPU 10–95%, memorie 1–128 MiB (ku te preset-i Data 256 MiB alokimi kufizohet në 64 MiB), si dhe *payload* rrjeti 1–1 024 KiB dhe dokumente 1-64 KiB, duke i refuzuar çdo kërkesë jashtë këtyre kufijve para ekzekutimit.

Së fundmi, procesi prind lexon progresin çdo sekondë, afatet mbikëqyren brenda shërbimit, kurse në rast ndalimi dërgohet sinjali përkatës duke ruajtur vetëm numëruesit e arritur deri në atë moment [1].

## 3.3. MongoDB, historiku dhe ndërfaqja

Për ruajtjen e të dhënave, MongoDB përdor databazën lab me koleksione të ndara (runs për përmbledhjet dhe workload_documents për ngarkesat e përkohshme), ku çdo rekord ruan ID-në, emrin, parametrat, shërbimet, kohët, gjendjen, numëruesit, gabimet, lidhjen e gjurmës dhe snapshot-in e vendosjes. Për performancë, indeksi sipas kohës ndihmon historikun, për dokumentet e përkohshme përdoret një indeks skadimi si mekanizëm ndihmës, ndërsa pastrimi normal bëhet pas përfundimit të punës [1].

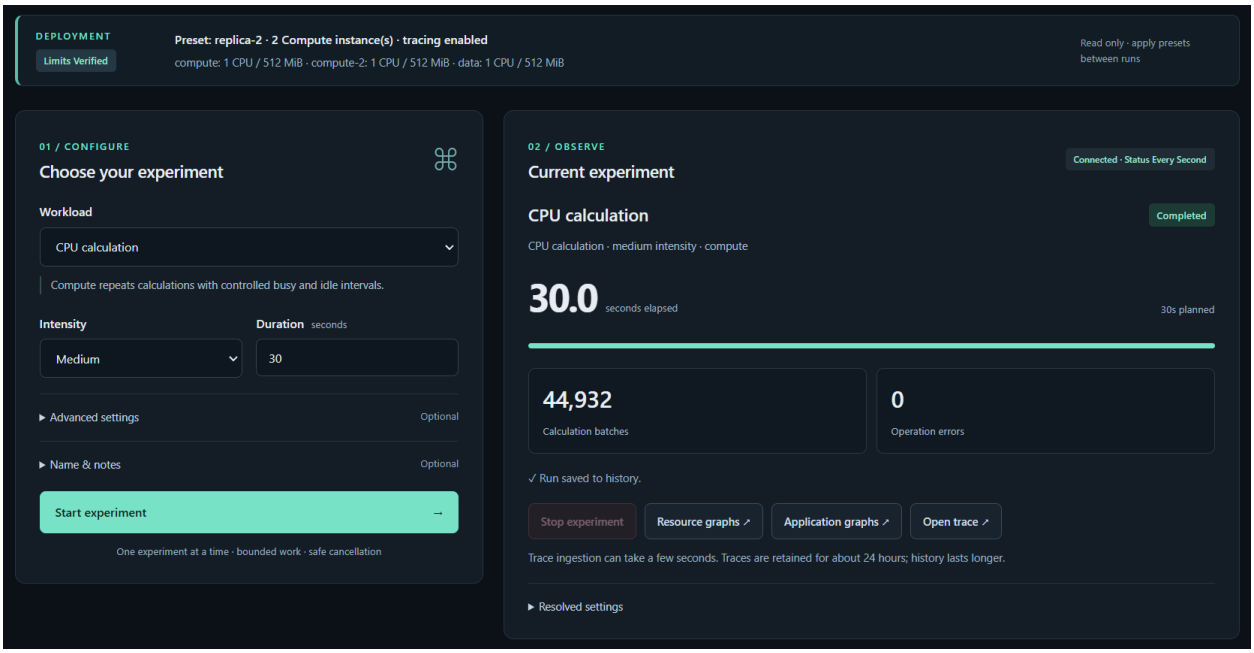



Figura 5: Paneli për konfigurimin dhe ndjekjen e eksperimentit

Ndërfaqja është ndërtuar me shabllone Flask, CSS dhe JavaScript të lehtë, duke pyetur për gjendjen një herë në sekondë në mënyrë që rifreskimi apo mbyllja e shfletuesit të mos e anullojë punën në server, ku CPU comparison tregon sasinë e punës dhe batch-et, ngarkesat e tjera paraqesin kohëzgjatjen, kurse ngjyrat shoqërohen gjithmonë me tekst për gjendjen, mirëmbajtjen dhe humbjen e lidhjes.

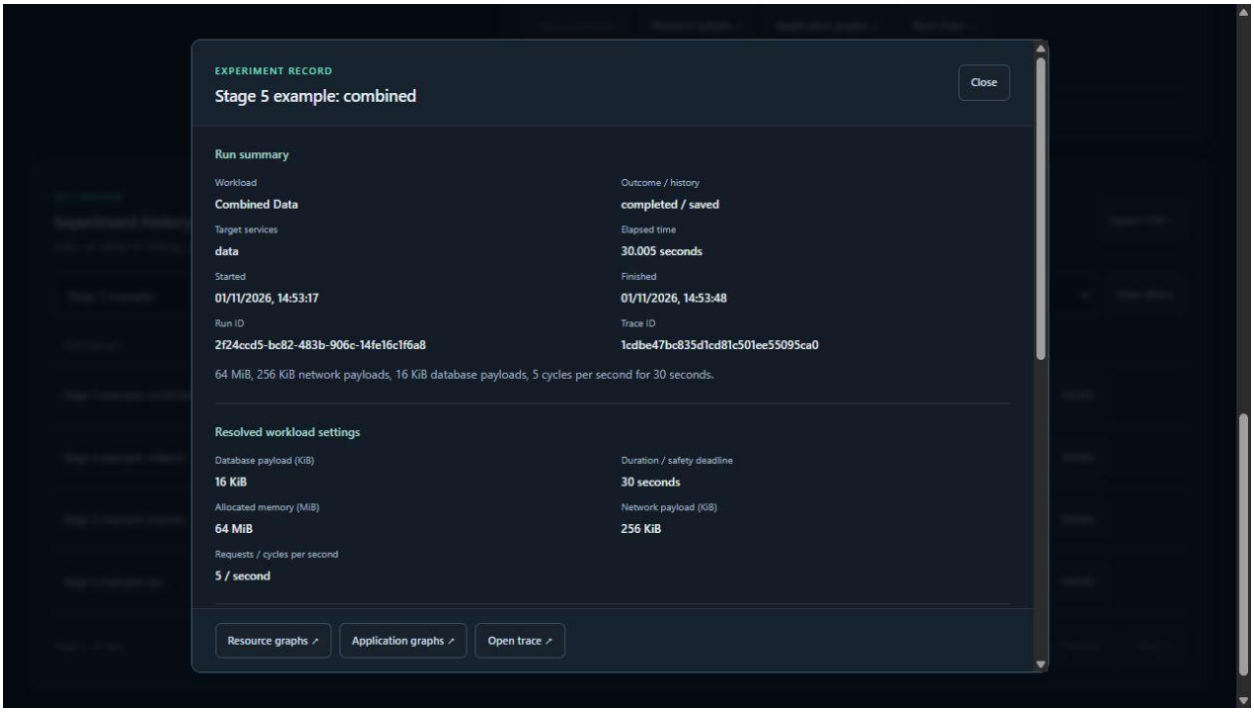



Figura 6: Parametrat dhe rezultatet e ruajtura të eksperimentit

Historiku mund të filtrohet sipas ngarkesës, gjendjes dhe emrit me faqe të ndara, eksporti CSV përdor po këto filtra për të ruajtur vlerat reale e jo vlerësime nga grafiku, lidhjet për Grafana përzgjedhin shërbimet dhe intervalin kohor (ku trace_id ruhet edhe nëse gjurma ka skaduar nga Tempo), ndërsa lokalisht historiku mund të inspektohet me Compass në 127.0.0.1:27017.

Së fundmi, nëse MongoDB mungon, shërbimet CPU, Memory dhe Network mbeten plotësisht të përdorshme ndërsa Database dhe Combined bëhen të padisponueshme, me Controller që mban deri në 100 përmbledhje të paruajtura në memorie duke riprovuar ruajtjen e tyre dhe duke dalluar qartë statusin saved nga pending.

## 3.4. Monitorimi dhe Grafana

Dashboard-i Container Resources përdor të dhënat e cAdvisor, të mbledhura çdo pesë sekonda. Etiketat e Compose identifikojnë projektin dhe shërbimin. Query-t kufizohen te cloud-lab1, përjashtojnë agregatet e host-it dhe kontejnerët e vjetër, si dhe shmangin numërimin e ndërfaqeve të dyfishta [1, 5].

Tabela 7: Kuptimi i matjeve të burimeve

| Matja | Njësia | Interpretimi |
|---|---|---|
| CPU e konsumuar | Bërthama | Norma e kohës CPU; vlera 1 përfaqëson një bërthamë të konsumuar. |
| Memory working set | MiB | Përdorimi i kontejnerit, përfshirë prindin, procesin e punës dhe bibliotekat. |
| Receive / transmit | Byte/s | Trafiku i ndërfaqes; përfshin overhead dhe aktivitet tjetër të shërbimit. |
| CPU / memory ndaj kufirit | % | Raporti me kuotën pozitive CPU ose kufirin e vlefshëm të memories. |
| Periudha të throttling | % | Pjesa e periudhave të planifikimit ku kufizimi CPU është zbatuar. |
| Kohë throttling | s/s | Normë kohëzgjatjeje; nuk është përqindje përdorimi CPU. |

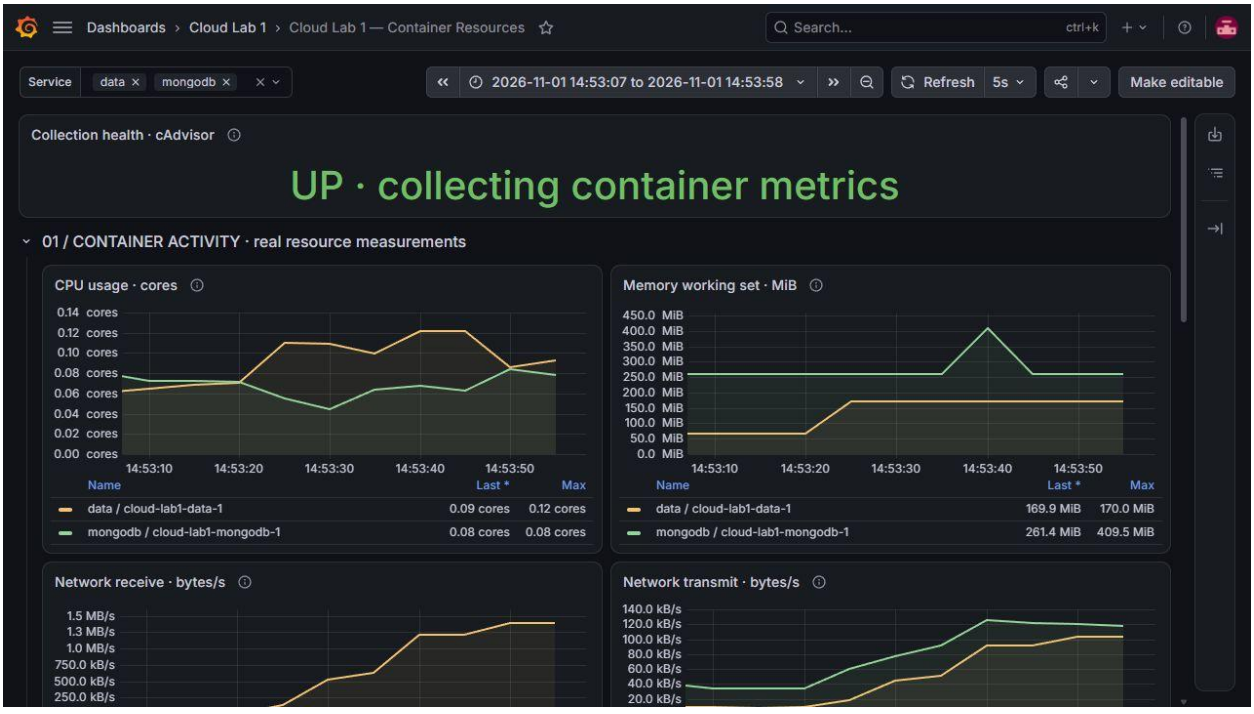



Figura 7: Dashboard-i i burimeve të kontejnerëve

CPU, memorja dhe trafiku i rrjetit e shfrytëzojnë dritaren prej 30 sekondash dhe përmes kësaj, për llogaritjen e ratës së ndryshimit, pra ndryshimi nga pika fillestare deri te ajo përfundimtare për atë interval kohor. Prandaj, ndryshimi shfaqet gradualisht dhe në pjerrtësi jo ekstreme njëjte

vazhdon të duket edhe fazën e përfundimit të punës anasjellatas. Seksioni i recent CPU përdor dy vlerat më të fundit të ingjestuara, zakonisht rreth pesë sekonda larg, dhe është tepër i përshtatshëm për punë të shkurtra, pasi kap vlerën e fundit. Këto dy pamje nuk duhen trajtuar si e njëjta matje vlerash.

CPU-ja ndaj kuotës dhe memoria ndaj kufirit shfaqen si të padisponueshme kur emëruesi mungon ose është i pakufizuar përshkak se në atë rast shfaqet gabimi i pjesëtimit me zero. Një zero reale ruhet si zero. Collection health tregon nëse Prometheus mund ta lexojë exporter, nuk provon që puna ose databaza janë funksionale. Për simulimin e trafikut të rrjetit, kemi Data service që është pranuesi i payload-eve dhe Compute dërguesi i tyre (një stream).

## Metrikat e aplikacionit dhe bashkimi i progresit

Dashboard Application Performance tregon dhe vizualizon punën e kryer ndërmjet burimeve të kontejnerëve. Çdo shërbim Flask mban registry në procesin prind ku ruan metrikat. Proceset e përkohshme dërgojnë numërues dhe bucket kumulative, prindi shton vetëm diferencën ndaj vlerave pra në metrik. Mesazhet e përsëritura ose përfundimtare nuk e numërojnë të njëjtën punë dy herë [1, 9].

Interpretimi bëhet më i dobishëm kur metrikat e aplikacionit gërshetohen dhe trajtohen së bashku me burimet e përdorura nga kontejnerët. Gjatë një ngarkese, pra një pune të ekzekutuar në CPU, aktiviteti i Compute krahasohet me raportin e llogaritjeve. Gjatë punëve trasnferi me rrjet ose databazë, observojmë numri i operacioneve, kohët e tyre dhe gabimet. Rritja e vonesës, e vetme, nuk përcakton shkakun e problemit, ajo duhet krahasuar me konsumin e burimeve dhe gjurmën përkatëse, nga aty ne mund të konsiderojmë hapat më intensiv dhe që shpenzojnë më së shumti kohë dhe sa resourse kemi ne dizpozicion gjatë ekzekutimit, pasi edhe kjo ndikon në shpejtësi. Në këtë mënyrë, dashboardat ndihmojnë për të shpjeguar marrëdhënien ndërmjet punës së kërkuar, punës së kryer dhe burimeve të përdorura.

Tabela 8: Familjet kryesore të metrikave të aplikacionit

| Familja e metrikave | Çfarë shpjegon |
|---|---|
| HTTP requests / HTTP duration | Numrin dhe vonesës sipas metodës, kërkesës dhe përgjigjes në route dhe statusit. |
| Runs active / runs total | Eksperimentin aktiv dhe të gjitha eksperimentet totale nga Controller. |
| Workload duration / elapsed | Kohën e ekzekutimit të worker-it dhe progresin gjatë punës. |
| Workload operations | Ciklet dhe ekzekutimet e batch nga CPU, përgjigjet, shkrimet/leximet dhe gabimet. |
| Network bytes | Byte-t e payload-it të lexuar nga Data, edhe kur transferimi ndërpritet. |
| I/O duration | Histogramet e kohës së transferimit ose operacionit të databazës. |

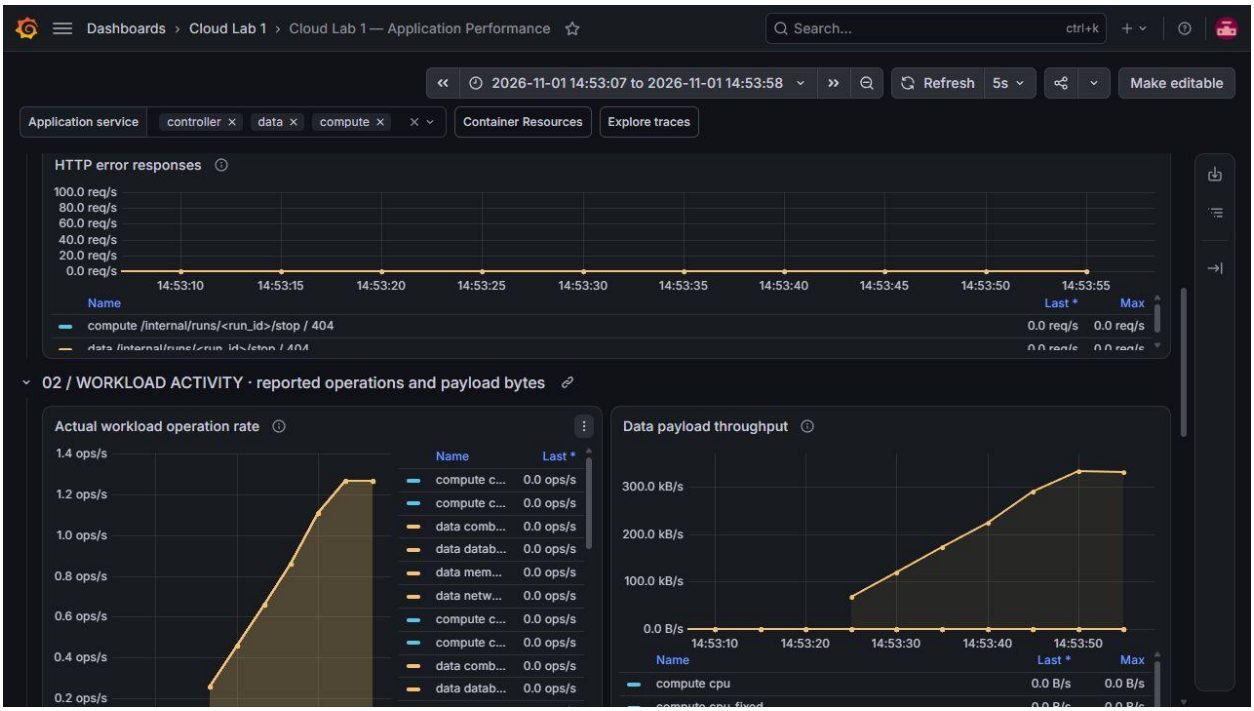



Figura 8: Dashboard-i i performancës së aplikacionit

Query për kërkesat dhe vonesën përdor dritare dyminutëshe, pasi kjo konsiderohet të jete observim me rrallë dhe duhet të jap interval më të gjatë. p95 është vlerësim nga bucket-et e histogramit, jo specifikisht koha e një kërkese aty. Kërkesat në endpointat si: /health, /metrics dhe metrika të tjera automatike nga libraritë të panevojshme për rastin konkret përjashtohen nga matjet HTTP. Kërkesat e zakonshme për statusin mbeten të dallueshme nga route.

ID-të, emrat e eksperimenteve dhe URL-të e papërpunuara nuk përdoren si etiketa në Prometheus. Përdoren modele dhe DTO si /api/runs/<run_id>, me vlera të kufizuara për llojin e punës, operacionin dhe rezultatin e pritur pastaj. Të gjitha metrikat humbin kur serviset janë të fikura. Por, MongoDB është databaza afatgjatë ku ruhen të gjitha punët e ekzekutuara. Përfundimi i eksperimentit numërohet një herë nga Controller, jo gjatë leximit të historikut ose përsëritjes së ruajtjes. [8]

Te cpu_fixed, etiketa historike cpu_batch numëron njësi llogaritëse, ndërsa logical_batch numëron batch-e të përfunduara prej 2 500 njësish. Me dy instanca, vlerat e matura mblidhen për Compute dhe Compute-2. Matjet e Grafana-s i përkasin shërbimit në intervalin e zgjedhur, ato nuk janë seri të izoluara për çdo run dhe mund të përfshijnë aktivitet paraprak pra graduale.

## 3.5. Gjurmimi i shpërndarë (Tracing)

Gjurmimi aktivizohet me profilin dhe me libraritë për të. OpenTelemetry instrumenton Flask, Requests dhe PyMongo, ndërsa spans të shprehura në kod përshkruajnë eksperimentin dhe fazat kryesore, mbrenda një gjurme janë pra sektorët. Inicializimi bëhet pas fork-ut të Gunicorn dhe veçmas brenda proceseve të punës. Konteksti ndërmjet serviseve që të dihet nga W3C kalon ndërmjet kërkesave HTTP, fijeve koordinuese dhe nisjes së procesit të përkohshëm. [1, 10, 11, 12]

Kërkesa Start është e shkurtër. Span-i experiment.run mbetet i hapur derisa të vëzhgohet një gjendje përfundimtare, ndërsa worker.execute dhe workload.run përshkruajnë punën në Data ose Compute. Për Combined, pasardhësit përfshijnë alokimin/mbajtjen e memories, kërkesat HTTP te Compute dhe thirrjet MongoDB nga klienti PyMongo. Secili nga këta hapa është span.

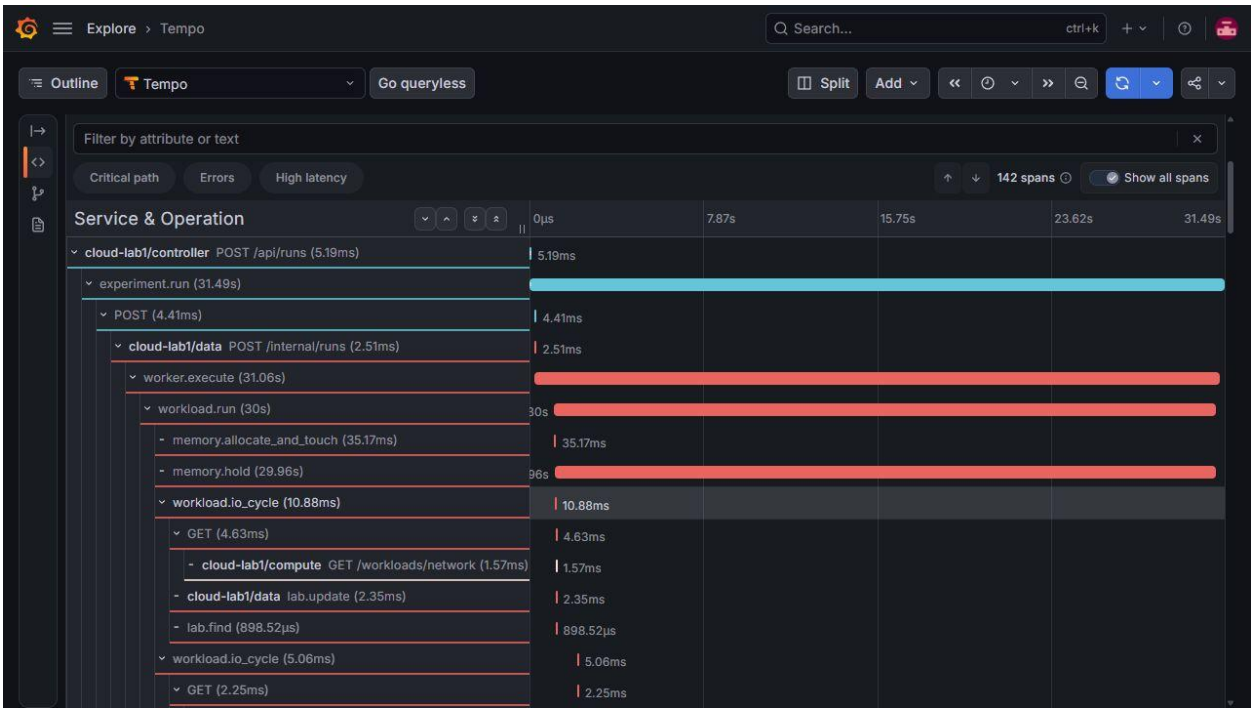



Figura 9: Gjurma e lidhur e eksperimentit Combined

Nuk duhet të marrim shumë zhurmë në traces dhe të mos bëhen gjigande, detajet e cikleve të suksesshme regjistrohen në ciklin e parë dhe më pas jo më shumë se një herë në sekondë. Çdo cikël i dështuar merr span ku është gabimi, edhe kur nuk ishte përzgjedhur për detaje, andaj gjithmonë evitohen gabimet në sistem kur ndodhin. Metrikat mbledhin të gjitha operacionet e raportuara pavarësisht nga gjurmët pasi ato mblidhen në një pipeline tjetër.

Gjendja failed ose interrupted shënohet si gabim, cancelled shënohet si ngjarje anulimi. Kontrollet e health, pings të disponueshmërisë dhe polling rutinë përjashtohen pasi janë të panevojshme dhe thjeshtë shkaktojnë zhurmë dhe shumë volum ta pa analizueshëm. Nuk kapen trupat e kërkesave/përgjigjeve ose përmbajtja e statement dhe komandave nga MongoDB sërish përshkak të volumit dhe zhurmës. Span-et MongoDB tregojnë kohën e thirrjeve të driver-it, jo fazat e brendshme të serverit apo ekzektuimit të planit të queryt.

Gjurma për punën Combined e ruajtur ka 142 span-e të lidhura pasi këtu ekzekutohet funksione nga shumë servise dhe ka trafik në të gjithë arkitekturën dhe është pjesa më diverse e funksioneve. Queue dhe timeout e eksportit janë të kufizuara, një Collector ose Tempo i padisponueshëm nuk duhet e ndryshon rezultatin e ngarkesës, pra nuk kemi ndonjë varshmëri direkte që do të dështonte sistemin kjo është thjeshtë një mbikqyrës. Një ndalim i fortë mund të humbë span-e dhe të lërë gjurmë jo të plotë. Skadimi i gjurmës pas rreth 24 orësh është i pavarur nga ruajtja e historikut. [1, 15]

ID e gjurmës: 1cdbe47bc835d1cd81c501ee55095ca0

## 3.6. Alokimi i burimeve dhe replikat

Presets ndryshojnë kufijtë dhe burimet e alokuara për instancat Compute dhe data gjithashtu disa presets edhe numrin e instancave Compute, mirëpo CPU comparison mban të njëjtën sasi pune. Një njësi kryen të njëjtën llogaritje deterministe mbi 3 000 numra, një batch logjik ka 2 500 njësi. Medium përdor 32 batch-e, pra 80 000 njësi, me checksum të përfunduar 12 007 132 800 000. Versioni i algoritmit është sum-squares-v1, realisht është një unazë e thjeshtë e cila ekzekuton shumëzim dhe mbledhje për një interval kohor ose për disa njësi pune të caktuara, afati që mos të ekzekutohet jobi për një kohë të gjatë është vendosur që të vihet limiti prej 180 sekondash [1] .

```python
"""An immutable assignment keeps work identical across limits and replicas."""
ALGORITHM = 'sum-squares-v1'
UNITS_PER_BATCH = 2500
UNIT_CHECKSUM = 150089160


def calculate_unit():  Florian Saqipi
    return sum(n*n % 104729 for n in range(3000))


def assign_batches(batch_count, instances):  Florian Saqipi
    slots_per_instance = 2 if len(instances) == 1 else 1
    plan = {service: [[] for _ in range(slots_per_instance)] for service in instances}
    for batch in range(batch_count):
        service = instances[batch % len(instances)]
        slot = (batch // len(instances)) % slots_per_instance
        plan[service][slot].append(batch)
    return plan
```

Figura 10: Shpërndarja deterministe e batch-eve

Me një Compute, dy procese marrin lista të alternuara batch pra të gjitha batches ekzekutohen në një instace. Me dy instanca, secila ka një proces dhe merr 16 nga 32 batch-et. Konfigurimi dhe ndarja bëhet para se shpërndarjes së batcheve të punës. Worker-i refuzon konfigurime të ndryshuara për të njëjtën ID; Controller pranon një përfundim për secilin batch dhe nuk e rishpërndan automatikisht punën e dështua.

Skriptat vendosin një flamur të qëndrueshëm mirëmbajtjeje dhe verifikojnë mungesën e një eksperimenti aktiv dhe bëjnë këto rregullime dhe thjesht vështrime se a janë të gjithat të dhënat prezente sipas strukturës. Pas ndryshimit, krahasojnë kufijtë e Docker-it me leximet cgroups v1/v2 të worker-ëve. Start lejohet vetëm pas verifikimit, pra nga ai konfigurim definohet nga presets resourset që do të përdorim. Snapshot i verifikuar ruhet me run-in e asoj pune dhe eksperimenti dhe nuk ndryshohet kur aplikohet një preset tjetër më vonë. Pra ai ruhet në atë kohë të ekzekutimit dhe pas asaj është gjithmonë read only. Kjo ruajtje bëhet në databazën e MongoDB. Pas kësaj rezultatet përsëri mund të vështrohen përmes UI dhe shihen te kontrolleri dhe

gjithashtu mund edhe të eksportohen nëse është nevoja në CSV file. Aty ka gjithashtu filtera sortim dhe kërkim për navigim më të lehtë dhe secila ruan një query të kostumizuar të klikueshëm si link në grafana dhe ruan ID të gjurmës gjithashtu. Kjo pastaj mund të shihet në grafana.

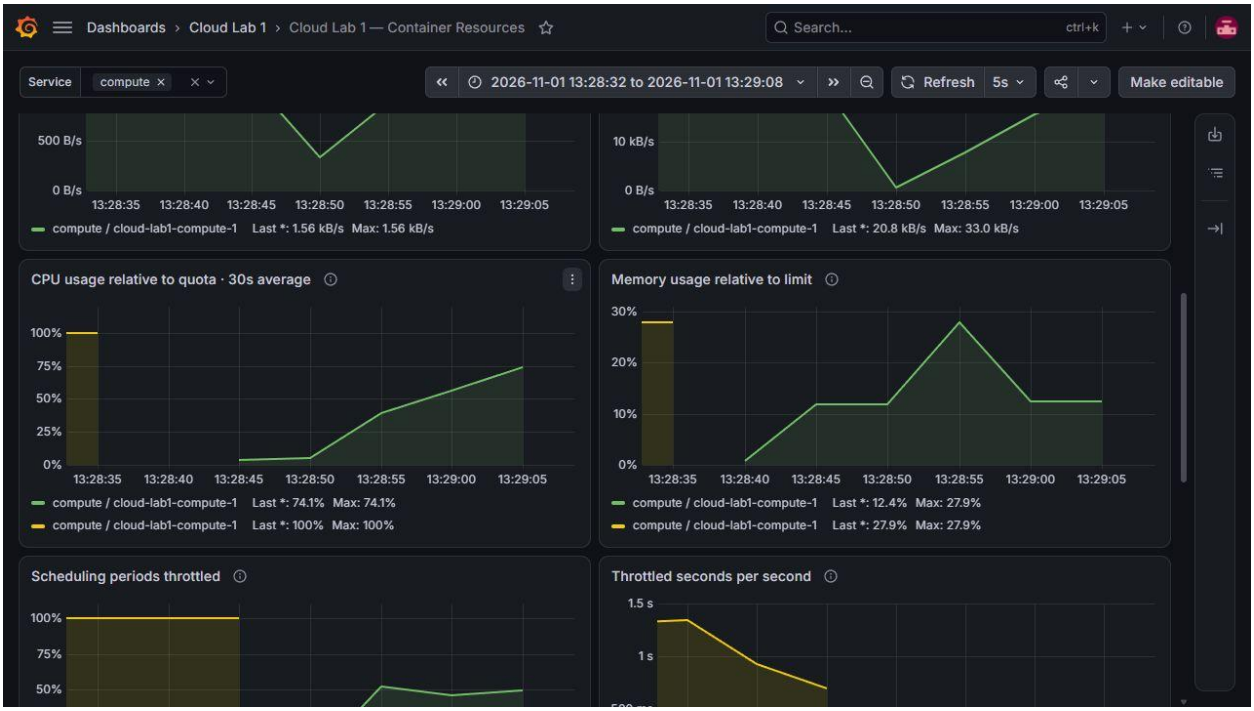



Figura 11: Përdorimi i CPU-së kundrejt kuotës

## Rezultatet e verifikimit të alokimit

Tabela 9 përmban ekzekutimet individuale nga eksperimentet konkrete të bëra dhe të dokumentuara. Të gjitha rastet përfunduan 32 batch-e me të njëjtin checksum dhe dy procese llogaritëse gjithsej. Koha është ajo e raportuar nga worker, limiti CPU përdor mostrat recent dhe nuk është maksimumi i mesatares 30-sekondëshe të shembujve të fazës 5. Mund të shohim se koha më e shkurtër eshtë me 2 cpu mirëpo dy instance me një CPU duhet pak më shumë kohë.

Tabela 9: Krahasimet CPU dhe një/dy instanca Compute

| Preset-i | Koha (s) | Kulmi recent CPU | Batch-et / proceset |
|---|---|---|---|
| cpu-05 | 68.643 | 0.502 bërthama | 32 në Compute; dy procese. |
| cpu-1 | 31.064 | 1.003 bërthama | 32 në Compute; dy procese. |
| cpu-2 | 14.884 | 1.929 bërthama | 32 në Compute; dy procese. |
| replica-1 | 31.922 | 1.001 bërthama | 32 në Compute; dy procese. |
| replica-2 | 15.664 | 0.998 / 1.000 | 16 për instancë; një proces secila. |




Figura 12: Burimet e instancave Compute dhe Compute-2

Te replica-2 rritet alokimi i përdorur në job total nga një CPU/512 MiB në dy CPU/1 024 MiB, pra edhe memorja rritet. Rënia e kohës nuk do të thotë të jetë ekzakt proporcionale, utilizimi luan rol këtu. Mostra e pavarur Docker arriti 1.971 bërthama të përdorura në rastin cpu-2, duke konfirmuar se konfigurimi mund të përdorë më shumë se një bërthamë kur pajisja e lejon.

Tabela 10: E njëjta ngarkesë Memory nën tre kufij

| Kufiri Data | Alokimi / koha reale | Kulmi working set |
|---|---|---|
| 256 MiB | 64 MiB / 30.001 s | 164.5 MiB |
| 512 MiB | 64 MiB / 30.001 s | 171.0 MiB |
| 1 024 MiB | 64 MiB / 30.001 s | 166.5 MiB |

Asnjë nga kontejnerët e vëzhguar nuk pati shumë problem sa i përket mungesës së memories. Kulmet nuk priten të jenë identike, sepse përfshijnë overhead dhe mostrim. Burimi i këtyre krahasimeve është nga eksperimentet e bëra me këta konteinera, bashkë me matjet dhe rekordet e veçanta për secilin preset. [15]

## 3.7. Eksperimentet dhe analiza e rezultateve

Katër shembujt e fazës u kryen njëri pas tjetrit e përdorëm presetin e baseline pra është default për arkitekturën dhe supporton gjithashtu gjurmimin, me imazhe të pandryshuara. Secili kishte të paktën 30 sekonda gjendje të qetë përpara dhe 30 sekonda ftohje pas punës, kur ekzekutoheshin eksperimentat pra. Mostrat Prometheus janë nga 5 sekonda larg sa merr pra puna e intervalit kur bëhet exportimi i metrikatve, raportet e burimeve janë mesatare 30-sekondëshe. Numëruesit e punës merren nga rezultati i ruajtur, jo nga forma e grafikut [15].

## Eksperimenti CPU

ID e run-it: 8636fafc-6533-44c2-a330-1cce68f0841a

Tabela 11: Parametrat dhe rezultati i eksperimentit CPU

| Elementi | Vlera e ruajtur |
|---|---|
| Profili / koha reale | Medium, 50% kohë aktive / 30.000 s |
| Puna / ritmi real | 44 953 cikle llogaritëse / 1 498.415 cikle/s |
| CPU Compute | Kulmi i mesatares 30 s: 0.558 bërthama |
| Gjendja / gabimet | completed, saved / 0 gabime |

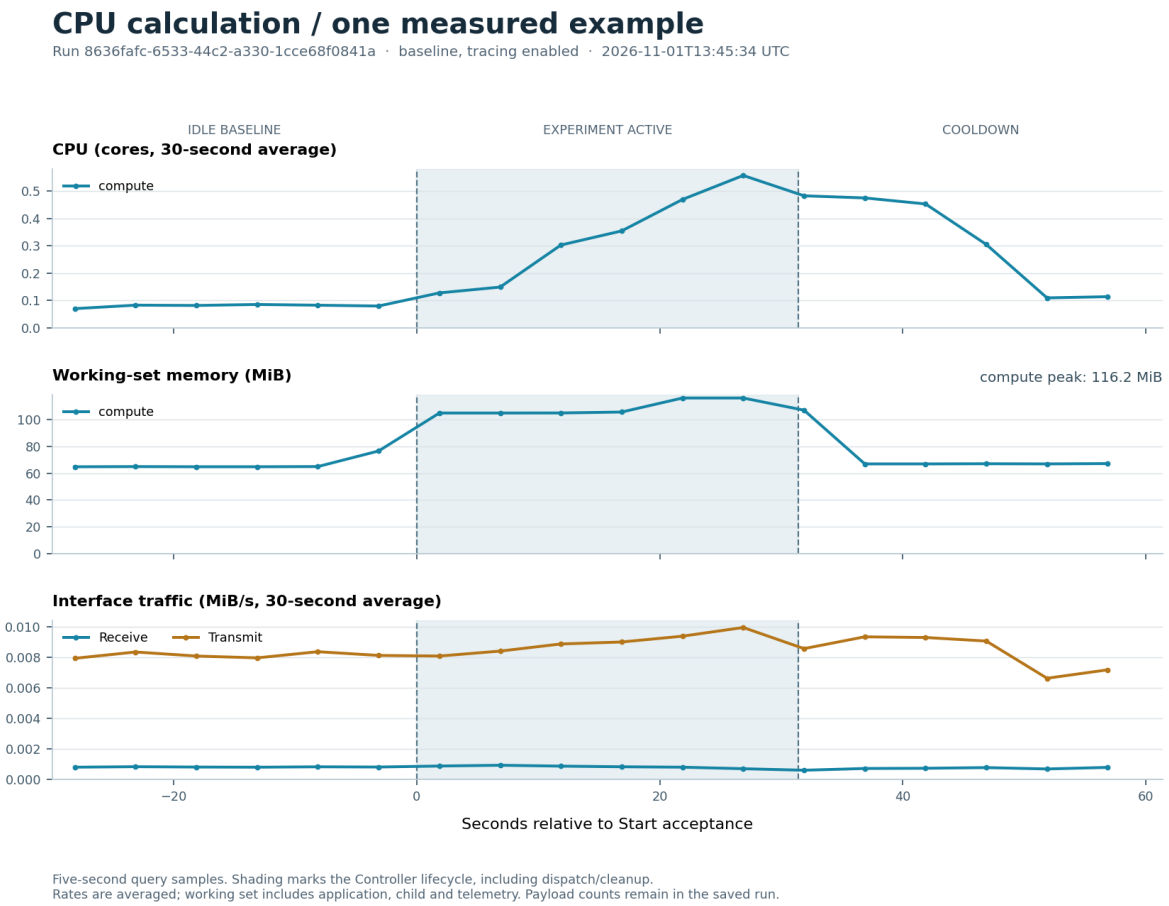



Figura 13: Matjet e eksperimentit CPU

Compute tregon rritje të aktivitetit kur puna fillon ekzekutimin ku mbahet CPU i nxënë për një intervalkohor dhe pastaj pëson një dhe rënie gjatë ftohjes. Vlera 0.558 përfshin punën e shërbimit dhe observabilitetit, prandaj nuk është verifikim i një përqindjeje ekzakte 50% dhe nuk mundet të jetë sepse ky kalkulim mund të mos jet ekzkat. Ciklet e kësaj ngarkese janë llogaritje elementare për një kohë ku CPU kalon në intervale të shkurte pune dhe jo pune,  nuk janë batch-et logjike prej 2 500 njësish të CPU comparison.

## Eksperimenti Memory

ID e run-it: 5a804b0d-a5c2-481a-992b-4e1e96536c92

Tabela 12: Parametrat dhe rezultati i eksperimentit Memory

| Elementi | Vlera e ruajtur |
|---|---|
| Buffer-i / koha reale | 64 MiB / 30.003 s |
| Faqet e prekura | 16 384 faqe prej 4 KiB |
| Working set Data | 65.8 → 169.1 → 66.0 MiB |
| Gjendja / gabimet | completed, saved / 0 gabime |

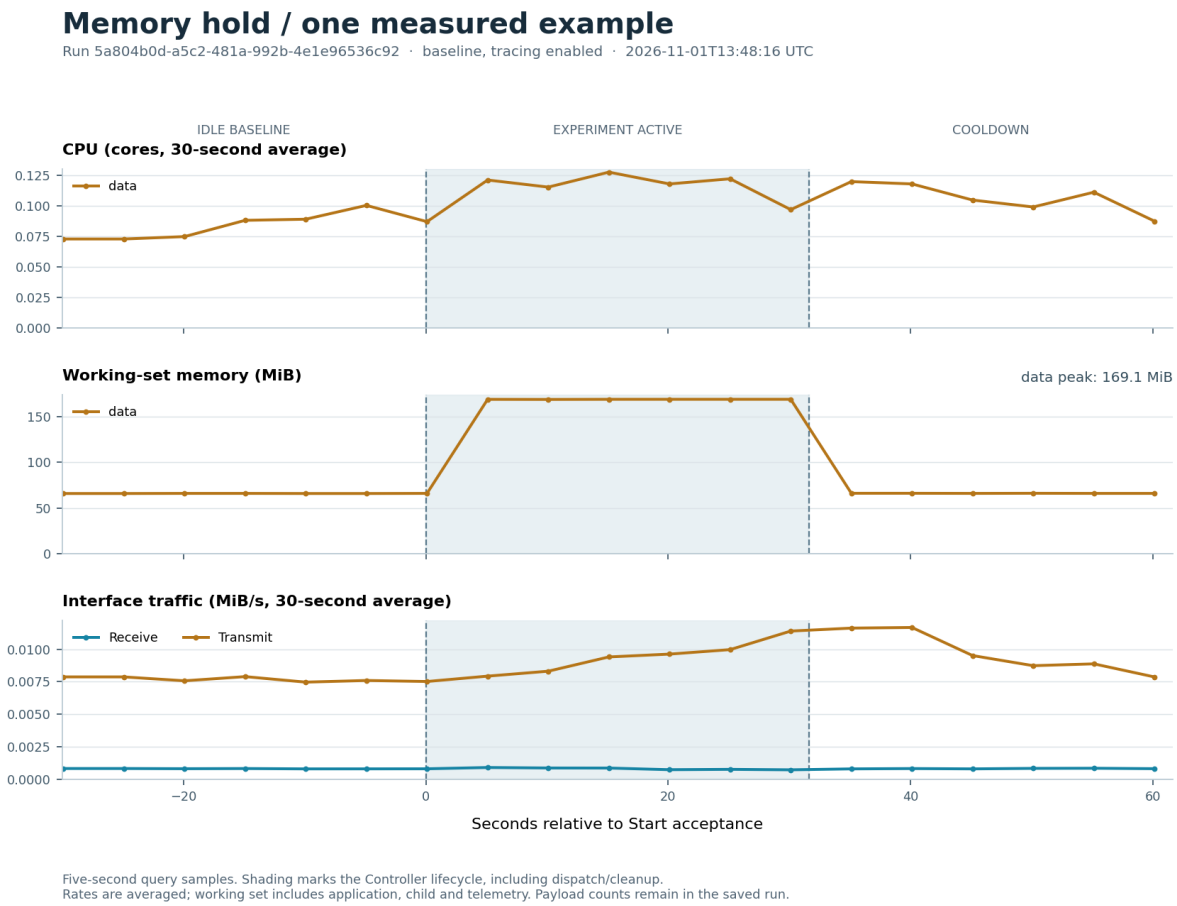



Figura 14: Matjet e eksperimentit Memory

Vlerat përmbledhin mesatarin e baseline-it, kulmin e mostrës së ruajtur dhe mesataret e fundit të ftohjes, kjo pra bëhet si kalkulim në dritare. Rritja prej rreth 103 MiB është më e madhe se buffer-i 64 MiB: bashkë me të niset interpreteri i procesit dhe bibliotekat e nevojshme. Kthimi afër 66 MiB mbështet lirimin e buffer-it pas daljes së procesit; nuk premtohet rikthim në të njëjtin numër byte-sh. Këtu gjithashtu duhet cekur se ka overhead të tjetër memorik se vet ai i cili kalkulohet si puna e bërë dhe ai gjithashtu ndikon në këtë aspekt

## Eksperimenti Network

ID e run-it: ead14011-9209-49a2-a061-9cb74ea41375

Tabela 13: Parametrat dhe rezultati i eksperimentit Network

| Elementi | Vlera e ruajtur |
|---|---|
| Payload-i / synimi | 256 KiB për përgjigje / 5 kërkesa/s |
| Koha / përgjigjet e plota | 30.000 s / 150 |
| Payload-i i lexuar / ritmi | 37.50 MiB / 5.000 kërkesa/s |
| Kulmi receive Data | 1.202 MiB/s, mesatare 30 s |
| Gjendja / gabimet | completed, saved / 0 gabime |

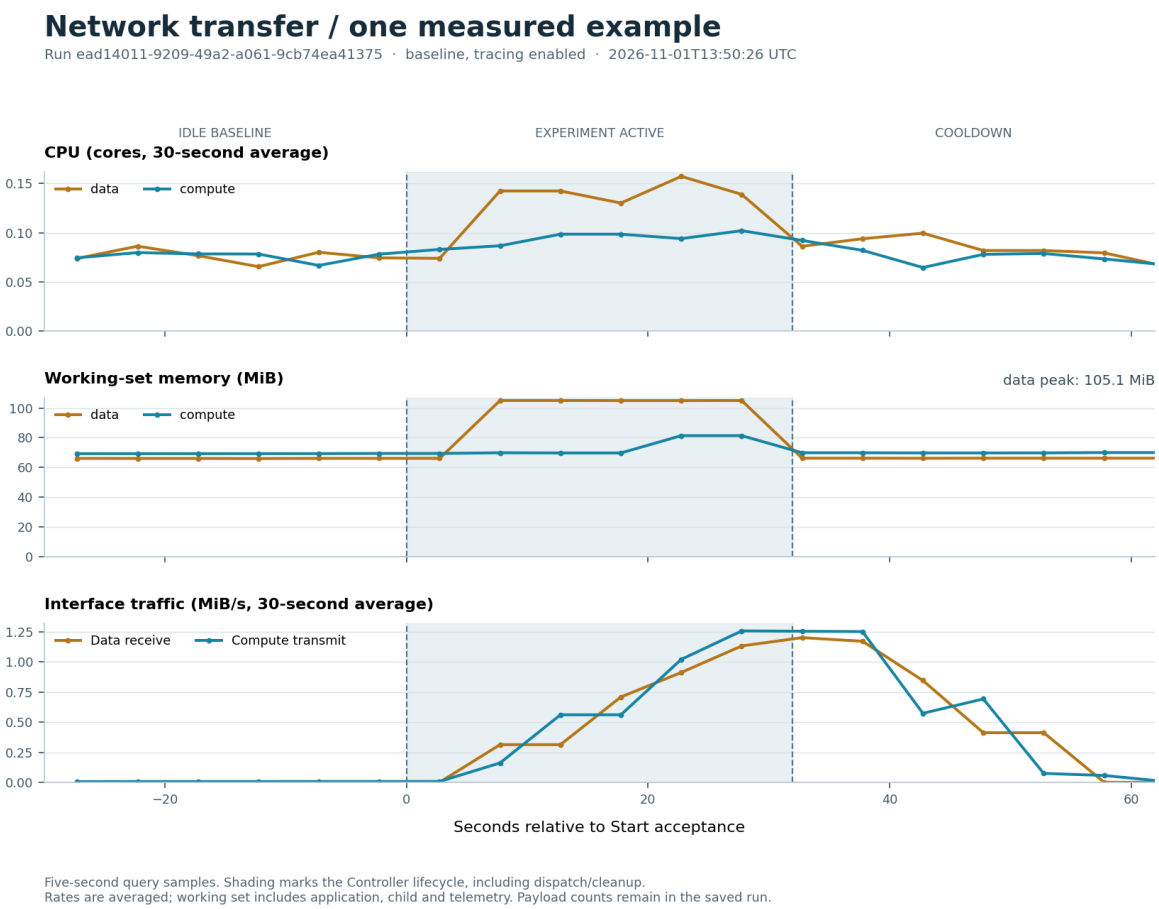



Figura 15: Matjet e eksperimentit Network

Numri 150 dhe sasia 37.50 MiB vijnë nga puna e këtij eksperimenti, dhe qëllimi i payload është 1.25 MiB/s, por kulmi në një grafik të mesatarizuar nuk është kontroll ekzakt i kësaj vlere, pasi ajo është një mesatare dhe trajton raportin e ndryshimit dhe jo vlerën ekzakte. Trafiku përfshin protokollet dhe kërkesa të tjera, ndërsa dritarja 30-sekondëshe mund të kapë edhe intervale para ose pas punës. Data pranon të dhëna, shkarkon. Compute i transmeton ato me stream.

## Eksperimenti Combined Data

ID e run-it: 2f24ccd5-bc82-483b-906c-14fe16c1f6a8

Tabela 14: Parametrat dhe rezultati i eksperimentit Combined

| Elementi | Vlera e ruajtur |
|---|---|
| Memoria / payload-et | 64 MiB; rrjet 256 KiB; databazë 16 KiB |
| Synimi / koha reale | 5 cikle/s / 30.005 s |
| Rrjeti | 150 përgjigje, 37.50 MiB, 4.999 kërkesa/s |
| Databaza | 150 shkrime, 150 lexime, 150 cikle; 4.999 cikle/s |
| Working set Data / gabimet | 66.3 → 170.0 → 66.4 MiB / 0 gabime |

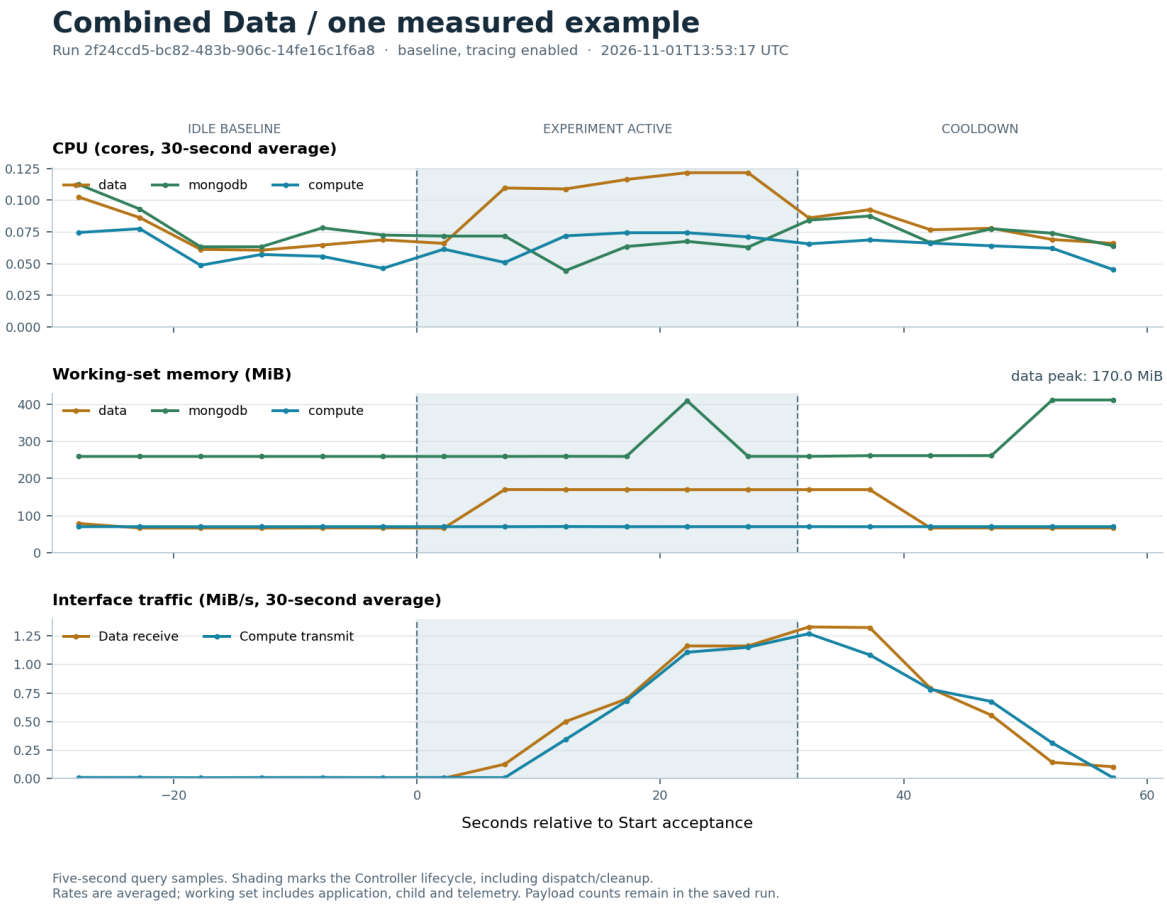



Figura 16: Matjet e eksperimentit Combined

Puna e kombinuar përfundoi dhe u ruajt, kjo punë ndorshin sepse përfshin memorien e Data-s, trafikun me Compute dhe shkrim/leximin MongoDB. Dokumentet e përkohshme fshihen, ndërsa përmbledhja mbetet në databazë. Kjo nuk garanton që cache i MongoDB të rikthejë menjëherë memorien në nivelin fillestar. Rata 4.999 pasqyron numëruesin dhe kohën reale, jo një dështim kundrejt synimit 5/s. Gjurma e këtij run-i është në Figurën 9 [4, 15].

## 3.8. Testimi, sfidat dhe kufizimet

Vlen të ceken edhe testet manuale të pranimit në mjedisin e punës. U bënë disa kombinime për teste manuale të cilat mbulojnë validimin, konfliktet, afatet, anulimin, proceset, rikuperimin dhe pjesët e observabilitetit. Disa nga kontrollet kontrollet e realizuar, përfundimtare të fazës përfshijnë query-t, CSV-në, gjendjet e ndërfaqes dhe materialet e ruajtura. Por gjithashtu edhe raste të tjera për sjelljen e sistemit nën ngarkesë dhe dështime parciale [15].

Tabela 15: Përmbledhje e verifikimeve të ruajtura

| Kontrolli | Rezultati i vëzhguar |
|---|---|
| Regresioni manual | Teste manuale të kaluara në kontejner të izoluar. |
| Kontrollet përfundimtare të fazës | Kontrolle të kaluara, të dhënat e mëparshme mbetën të disponueshme edhe në dështime të pjesshme të bëra manualisht. |
| Katër shembujt | Të përfunduar, të ruajtur, pa gabime operacionale të raportuara. |
| Stop nga ndërfaqja, Memory | Cancelled ekzekutohet brenda 1.958 s në rastin e testuar. |
| Stop me replika, faza 4 | Për vetëm 1.054 .s pa procese llogaritëse të mbetura. |
| Gjendja përfundimtare e fazës | Baseline me gjurmim, me nëntë shërbime me asnjë run aktiv, rezultat pending ose dokument i përkohshëm. Pra vetëm rezultatet tregohen në këtë gjendje pasive |

Rifreskimi i browserit ruan dukshmërinë e punës aktive kjo pasi kontrolleri si aplikacion servon rezultatet e ruajtura nga databaza. Kur kemi pjesë të arkitekturës jo funksionale apo gabim Start butoni është disabled me mesazh shpjegues. Restart-et e Controller ose worker-it shënuan interrupted dhe nuk e përsëritën punën. Worker i paarritshëm e mban sistemin të bllokuar derisa të përcaktohet gjendja e tij, pra është një lock që mos të ndodh ndonjë lloj i race condition mbi këtë.

Gjatë implementimit u vërejt se querit në disa pika nuk tregonin të dhëna përshkak se dritaret kan qenë shumë të shkrutëra qe të kapin normën e ndryshimit dhe kjo pastaj kërkon pak kalibrim të dritareve në query dhe refreshit. Pra u korrigjuan query që përfshinin seri, sintaksa e bashkimit të raporteve dhe përdorimi i mesatares 30-sekondëshe për punë që zgjat më pak. U bënë për verfikim mandej kontrolle për mostrat e reja dhe krahasim me Docker stats. Këto hapa tregojnë pse një dashboard bosh ose një exporter i arritshëm nuk mjafton si provë e monitorimit korrekt 100%.

Matjet dhe metrikat përfshijnë Python, kontrollet e health dhe observabilitetin. Mostrimi pesësekondësh mund të humbë limite të mbërritura të shkurtra, dritaret 30-sekondëshe dhe dyminutëshe e zbusin sinjalin mirëpo mund të mos jet aq responsiv. p95 me pak vëzhgime është i përafërt për kërkesat. Numëruesit resetohen në restart dhe një proces i ndalur humbë intervalin e fundit të progresit. Gjurmët kanë kohë ruajtjeje më të shkurtër se historiku.

Katër eksperimentet e bëra dhe krahasimet e alokimit janë observime individuale në një makinë të vetme, me ngarkesa pra të simuluara koj dihet. Nuk supozohet të japin një interval besueshmërie, performancë të garantuar ose efikasitet shkallëzimi me burime të barabarta. Burimet dhe ID-të e run-eve lejojnë kontrollimin e numrave dhe gjithashtu metrikat.

# IV. Përfundime

## 4.1. Arritjet kryesore

Projekti ka zhvilluar dhe ka përmbushur kërkesën për monitorimin e CPU-së, memories dhe rrjetit të kontejnerëve përmes Prometheus dhe Grafana, mirëpo shkon edhe më tutje me observabilitet. Serviset me Flask krijojnë ngarkesa apo punë në njësi dhe resourse të caktuara, ndërsa cAdvisor ndegjon dhe merr matje në ndikimin e tyre në infrastrukturë. Fillmisht mund të konfigurohet eksperimenti infrastruktura dhe resurset, fillohet puna dhe observohet progresi. Visualizimet në grafike kështu i aplikohen parametrat dhe veprimin që e shkaktuan.

Numëruesit nëpër servise si metrika masin punën e kryer dhe gabimet, gjurma Combined lidh Controller, Data, Compute dhe thirrjet e klientit MongoDB, pra komplet nga një pjesë nga infrastruktura. Historiku, eksporti CSV dhe konfigurimet e ruajtura të eksperimenteve mundësojnë kontrollimin e rezultateve. Kontrollet e dokumentuara mbështesin funksionimin e anulimit, pastrimit dhe rikuperimit brenda rasteve të verifikuara.

## 4.2. Gjetjet dhe kufizimet

Katër eksperimentet e realizuara dëshmojnë ndryshime në burimet përkatëse, CPU e rriti aktivitetin e Compute, alokimi 64 MiB e rriti memorien e Data-s dhe dalja e procesit e afroi atë me nivelin fillestar. Network përfundoi 150 përgjigje, me 37.50 MiB payload, ndërsa Combined përfundoi edhe 150 cikle shkrim/leximi në databazë. Memoria totale e kontejnerit përfshin edhe proceset dhe overheadin tjetër nga serviset e libraritë, andaj nuk barazohet me buffer. Fshirja nuk garanton lirimin e menjëhershëm të cache nga memorja internale e MongoDB [15].

Krahasimet CPU me të njëjtën punë dhe checksum treguan kohë më të shkurtër kur u rrit kuota: nga 68.643 sekonda me 0.5 CPU në 14.884 sekonda me 2 CPU. Me dy instanca Compute, puna u nda duke mbajtur dy procese llogaritëse gjithsej. U rritën edhe CPU-ja dhe memoria totale e lejuar; ky rezultat nuk provon efikasitet shkallëzimi me burime të barabarta. Buffer-i 64 MiB u përdor me sukses nën të tre kufijtë e memories të testuar [15].

Secila prej eksperimenteve janë observime individuale në një makinë me ngarkesa të gjeneruara statike si punë. Mostrimi pesësekondësh dhe natyra e querieve të veglave për metrika dhe vizualizim zbusin kulmet dhe ta vonojnë paraqitjen e ndryshimit në bazë të konfigurimeve. Matjet përfshijnë punën të aplikacionit dhe observabilitetit. Rezultatet demonstrojnë mekanizmat, dhe performancën gjenerale të sistemit dhe arkitekturës.

## 4.3. Zhvillimi i ardhshëm

Hapi vijues është një studim i ngjashëm, mirëpo me fokus ne rend të kontrolluar të konfigurimeve të zgjeruara dhe intervale të njëjta, në këtë rast do të ishte i mundur krahasimi i ndryshueshmërisë së kohëve dhe ndikimit të gjurmimit në sistem dhe resourse. Zgjerimet mund të përfshijnë ngarkesa nga aplikacione reale, alarme dhe vendosje në makina virtuale ose cloud publik. Parametrat dhe rezultatet e ruajtura ofrojnë bazën për këto prova të kontrolluara.

# V. References

Electronic sources were accessed on November 1, 2026. Project source code and configuration files are listed in [1]; verification and experiment evidence is listed in [15].

[1] Project Group, "CRM_GR5_observability_stack," GitHub, 2026. Accessed: Nov. 1, 2026. [Online]. Available: https://github.com/floriansaqipi/CRM_GR5_observability_stack

[2] Docker Inc., "Docker Compose," Docker Documentation. Accessed: Nov. 1, 2026. [Online]. Available: https://docs.docker.com/compose/

[3] Pallets Projects, "Gunicorn," Flask Documentation. Accessed: Nov. 1, 2026. [Online]. Available: https://flask.palletsprojects.com/en/stable/deploying/gunicorn/

[4] MongoDB Inc., "WiredTiger Storage Engine," MongoDB Manual. Accessed: Nov. 1, 2026. [Online]. Available: https://www.mongodb.com/docs/manual/core/wiredtiger/

[5] Prometheus Authors, "Monitoring Docker container metrics using cAdvisor," Prometheus Documentation. Accessed: Nov. 1, 2026. [Online]. Available: https://prometheus.io/docs/guides/cadvisor/

[6] Prometheus Authors, "Overview," Prometheus Documentation. Accessed: Nov. 1, 2026. [Online]. Available: https://prometheus.io/docs/introduction/overview/

[7] Grafana Labs, "Provision Grafana," Grafana Documentation. Accessed: Nov. 1, 2026. [Online]. Available: https://grafana.com/docs/grafana/latest/administration/provisioning/

[8] Prometheus Authors, "Metric and label naming," Prometheus Documentation. Accessed: Nov. 1, 2026. [Online]. Available: https://prometheus.io/docs/practices/naming/

[9] Prometheus Python Client Authors, “Custom Collectors,” Prometheus Python Client Documentation. Accessed: Nov. 1, 2026. [Online]. Available: https://prometheus.github.io/client_python/collector/custom/

[10] OpenTelemetry Authors, “OpenTelemetry Flask Instrumentation,” OpenTelemetry Python Contrib Documentation. Accessed: Nov. 1, 2026. [Online]. Available: https://opentelemetry-python-contrib.readthedocs.io/en/latest/instrumentation/flask/flask.html

[11] OpenTelemetry Authors, “Propagation,” OpenTelemetry Documentation. Accessed: Nov. 1, 2026. [Online]. Available: https://opentelemetry.io/docs/languages/python/propagation/

[12] OpenTelemetry Python Authors, “Working With Fork Process Models,” OpenTelemetry Python Documentation. Accessed: Nov. 1, 2026. [Online]. Available: https://opentelemetry-python.readthedocs.io/en/latest/examples/fork-process-model/README.html

[13] OpenTelemetry Authors, “Collector,” OpenTelemetry Documentation. Accessed: Nov. 1, 2026. [Online]. Available: https://opentelemetry.io/docs/collector/

[14] Grafana Labs, “Grafana Tempo,” Grafana Tempo Documentation. Accessed: Nov. 1, 2026. [Online]. Available: https://grafana.com/docs/tempo/latest/

[15] Project Group, “Stage 4-5 verification and experiment evidence,” GitHub, 2026. Accessed: Nov. 1, 2026. [Online]. Available: https://github.com/floriansaqipi/CRM_GR5_observability_stack/tree/main/docs/evidence